

★ Get unlimited access to the best of Medium for less than \$1/week. [Become a member](#)



End-to-end Azure data engineering project Part 2: Implementing medallion architecture in Azure Databricks



ch geetanjali

Follow

3 min read · Apr 30, 2024



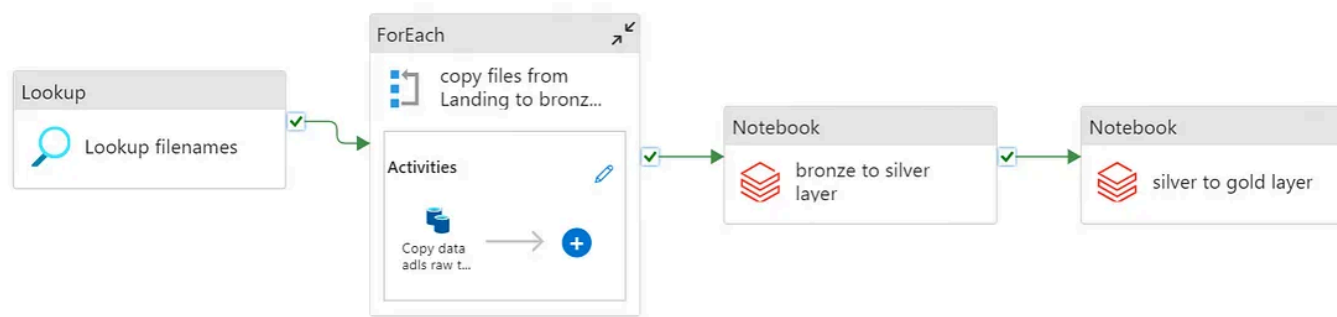
28



1



In the previous blog we saw how to ingest the data from Landing folder using Azure data factory to the bronze layer. In this blog we will see how we incrementally progress from bronze to silver layer.



Azure Data Factory pipeline

Prerequisites:

1. Create a Databricks workspace from Azure portal. After launching the workspace create a cluster (single node cluster with Standard F4 node type will be sufficient).
2. Mount the ADLS Gen2 storage to Azure Databricks using below source code:

```
mounted_filesystems = dbutils.fs.mounts()
mount_point_to_check = "/mnt/medallion"
is_mounted = False
for mount in mounted_filesystems:
    if mount.mountPoint == mount_point_to_check:
        is_mounted = True
        break
if is_mounted:
    print("Mount point is mounted.")
else:
    print("Mount point is not mounted.")
    dbutils.fs.mount(
        source="wasbs://<container_name>@<storage_account_name>.blob.core.windows.net",
        mount_point="/mnt/medallion",
        extra_configs={
            "fs.azure.account.key.<storage_account_name>.blob.core.windows.net": "<storage_account_key>"
        }
    )
```

Data Transformation using Azure Databricks:

Create schema and parquet tables in bronze layer:

1. Before proceeding create a dedicated schema for bronze layer tables . We need to iterate over the files present in the bronze layer and need to create tables out of it. You can run the below code snippet to do so:

```

table_names = []
for file_info in dbutils.fs.ls('/mnt/medallion/data/bronze'):
    table_name = file_info.name.split(".")[0].rstrip('/')
    table_names.append(table_name)

for i in table_names:
    source_path = "/mnt/medallion/data/bronze/"+i+'.parquet'
    source_df = spark.read.format('parquet').load(source_path)
    table_schema = source_df.schema
    target_table_location = f"/mnt/medallion/data/bronze/{i}.parquet"
    table_schema = ", ".join([f"{field.name} {field.dataType.typeName()}" for fi

    create_table_sql = f"""
    CREATE TABLE IF NOT EXISTS {i} ({table_schema})
    USING parquet
    LOCATION '{target_table_location}'
    """

    spark.sql(create_table_sql)

```

Create schema and delta tables in silver layer:

1. Before proceeding create a dedicated schema for silver layer tables .
Iterate over the files present in the bronze layer and create tables with the same schema in the silver layer. You can run the below code snippet to do so:

```
from pyspark.sql.functions import date_format, from_utc_timestamp
from pyspark.sql.types import TimestampType

for i in table_names:

    #source details
    source_path = "/mnt/medallion/data/bronze/"+i+'.parquet'

    #Remove duplicates in bronze layer
    source_df = spark.read.format('parquet').load(source_path)
    source_df = source_df.drop_duplicates()
```

Medium Search Write

```
# Convert the schema to string representation
table_schema = ", ".join([f"{field.name} {field.dataType.typeName()}" for fi

create_table_sql = f"""
CREATE TABLE IF NOT EXISTS {i} ({table_schema})
USING delta
LOCATION '{target_table_location}'
"""
spark.sql(create_table_sql)
```

2. We first create a staging table from the bronze layer tables before uploading the data to the silver layer tables. We then merge the data from bronze layer tables to silver layer tables using the below code snippet.

```
for i in table_names:
    source_table_df = spark.sql(f"SELECT * FROM bronze.{i}")
    target_table_df = spark.sql(f"SELECT * FROM silver.{i}")
    first_column_name = target_table_df.columns[0]

    # Get column names from the target table
    target_table_columns = target_table_df.columns

    # Create temporary views for staging in bronze layer before uploading to silver
    source_table_df.createOrReplaceTempView("source_temp_view")
    spark.sql("SELECT * FROM source_temp_view")

    # Merge the results to silver layer tables
    merge_sql = f"""
    MERGE INTO silver.{i} AS tgt
    USING (
        SELECT *,
            ROW_NUMBER() OVER (PARTITION BY {first_column_name} ORDER BY {first_
        FROM source_temp_view
    ) AS src
    ON tgt.{first_column_name} = src.{first_column_name} AND src.row_num = 1
    WHEN MATCHED THEN
        UPDATE SET {'', ''.join([f"tgt.{col} = src.{col}" for col in target_table_
    WHEN NOT MATCHED THEN
        INSERT ({' ', ''.join(target_table_columns)})
        VALUES ({' ', ''.join([f"src.{col}" for col in target_table_columns])});
    """
    spark.sql(merge_sql)
```

3. After loading the data in the silver layer we require to perform further transformations to be ready for different use cases. Thus, we performed transformation from data in the silver layer by trimming the time from the timestamp column in one of the silver layer tables. This transformation was required to set the granularity of the data in the gold layers to date(which does not include time). The transformed data was then stored in different table in the silver layer(You can replace the table in the silver layer if you want).

```
from pyspark.sql.functions import col, regexp_replace

df = spark.sql("select * from bronze.olist_orders_dataset")
dfp_trimmed = df.withColumn("trimmed_purchase_date",col("order_purchase_timestamp"))
dfp_trimmed=dfp_trimmed.withColumn("trimmed_purchase_date_replace",regexp_replace(col("trimmed_purchase_date"),"T.*",""))
dfp_trimmed.createOrReplaceTempView("olist_orders_dataset_silver_stg")

#create a staging table in silver layer for orders table
df = spark.sql ("select * from olist_orders_dataset_silver_stg")
dfp_trimmed.write.mode("overwrite").format("delta").save("/mnt/medallion/data/silver/olist_orders_dataset_modified.d")

table_schema = dfp_trimmed.schema
table_schema = ", ".join([f"{field.name} {field.dataType.typeName()}" for field in table_schema.fields])

#create a new table for olist_orders_dataset_modified
create_table_sql = f"""create table if not exists silver.olist_orders_dataset_modified
using delta location '/mnt/medallion/data/silver/olist_orders_dataset_modified.d'"""
```

```
spark.sql(create_table_sql)
```

After completing the implementation of bronze to silver layer notebook you can test the pipeline using debug option or trigger option in ADF. The successful completion of pipeline will result in delta files in the silver layers of the medallion architecture . Now the next step will be to further transform the data to ingest in the gold layer .

Databricks

Data Engineering

Medallion Architecture

Big Data Analytics

Azure

**Written by ch geetanjali**

72 followers · 13 following

Follow

Responses (1)



majid aminian

What are your thoughts?



lam Aarif Azure

May 30, 2024



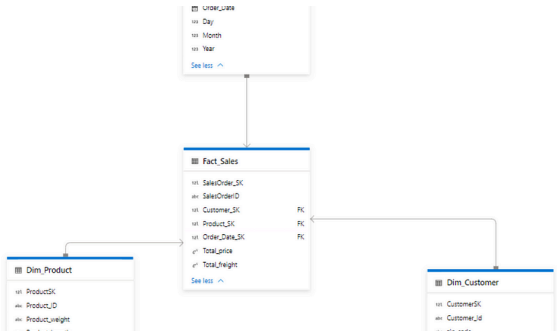
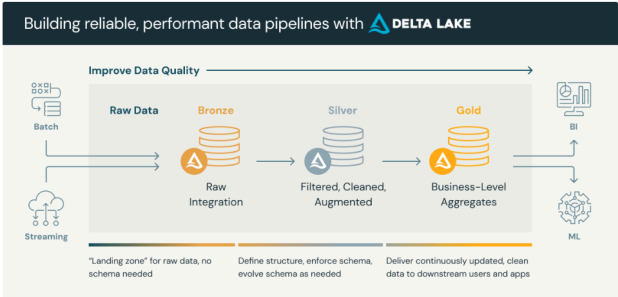
When you are posting the final part



1 reply

Reply.

More from ch geetanjali



 In Dev Genius by ch geetanjali

End-to-end Azure data engineering project Part 1:...

This blog comprises of implementing Medallion architecture in Azure Databricks....

Apr 24, 2024  14



 ch geetanjali

End-to-end Azure data engineering project Part 3:...

In the previous blog we saw data flow from bronze to silver layer using Azure Databrick...

Jun 5, 2024  16



See all from ch geetanjali

Recommended from Medium

Feature	Benefit
Declarative CDC	No complex logic or orchestration needed
Full Type 2 Support	Keeps full historical records
Real-Time Updates	Enables up-to-date dashboards and ML models
Flexibility in Handling Deletes/Truncates	Meets varying data semantics
Simplified Code	Declarative syntax vs. thousands of lines of code



 In Towards Data Enginee... by THE BRICK LEARN...

Databricks 2025 : Simplifying Real-Time Data Pipelines with AUTO...

In today's world of ever-changing data, Change Data Capture (CDC) is no longer a...

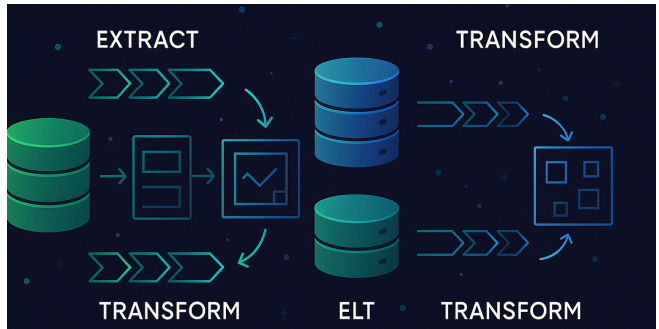
★ Jun 13 🖱️ 17  ...

 Nnaemezue Obi-Eyisi

Databricks Just Changed the Game with LakeFlow—But Is It Too Sma...

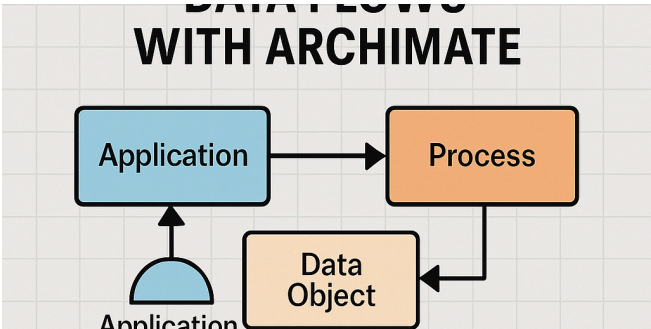
It's not often that a new tool makes me stop mid-scroll and say, "Wait... this changes..."

★ Jun 15 🖱️ 53 💬 2  ...



 Code With Hannan

ETL vs ELT: Design Patterns for Modern Data Infrastructure



 Armando Vázquez

Best Practices for Documenting Data Flows with ArchiMate

In the era of big data, building a robust and scalable data pipeline is a fundamental need...

A Practical Guide for Mapping Information Movement Across Systems and Processes

★ Jun 16 🖱 13 📌 ⋮

★ Jun 6 💬 1 📌 ⋮



 Victor Oketch Sabare

Stop Learning Pandas: The Spark 4.0 Features Nobody’s Telling You...

Apache Spark 4.0 finally lets Python-first data folks skip the “collect-to-Pandas” detour. A...

★ Jun 10 🖱 37 💬 2 📌 ⋮

 Anubhav

PySpark Interview Prep Day 5: Understanding Resilient...

Introduction

★ Feb 8 🖱 1 📌 ⋮

See more recommendations

[Help](#) [Status](#) [About](#) [Careers](#) [Press](#) [Blog](#) [Privacy](#) [Rules](#) [Terms](#) [Text to speech](#)